Project Report: GYMSystem Management Platform

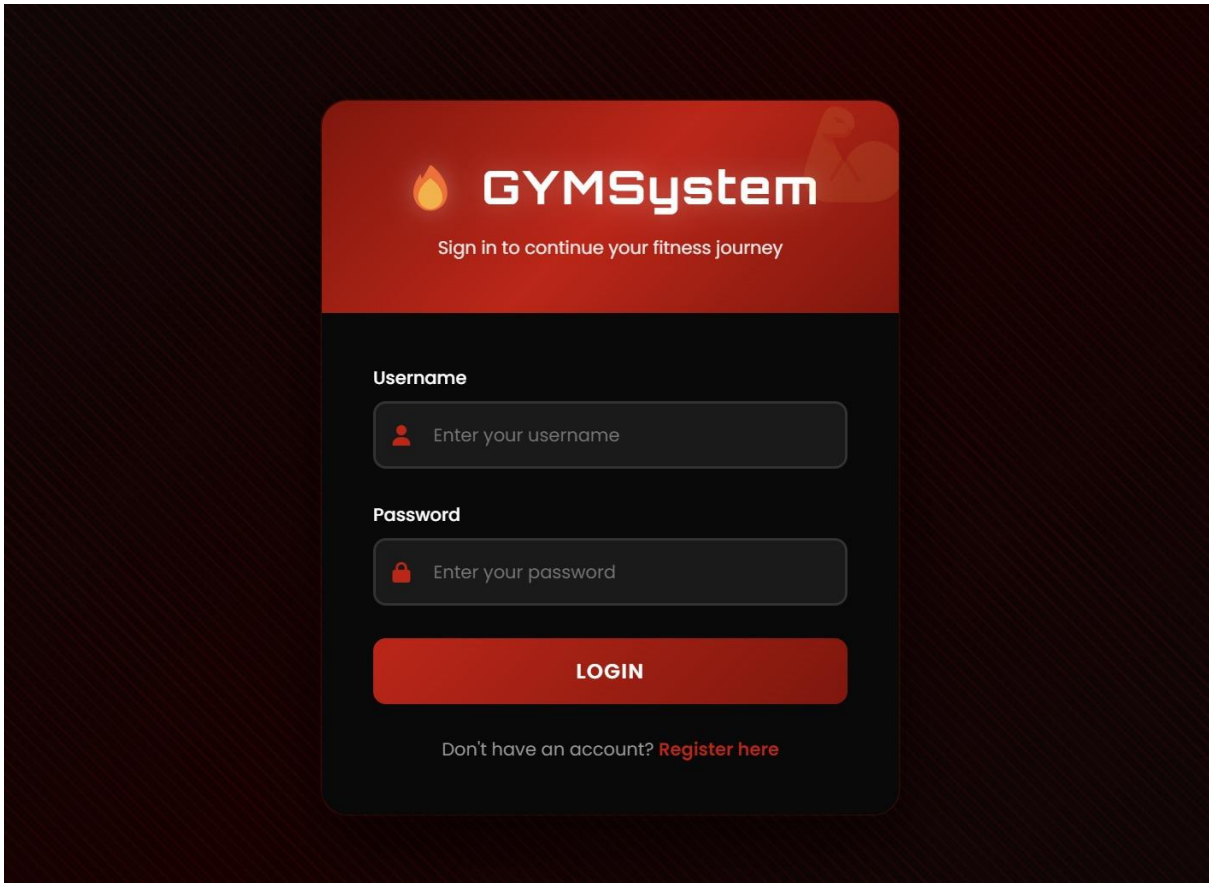
Project Name: GYMSystem Management Platform

The Core Idea: GYMSystem is a centralized management portal designed for modern fitness centers. It bridges the gap between gym administration and the end-user by providing a role-based interface. The system automates the booking of fitness sessions, membership tracking, and equipment status monitoring, moving away from manual paper-based logs to a data-driven digital environment.

2. Features Implemented

- **Role-Based Access Control (RBAC):** Distinct dashboards and permissions for Admins, Trainers, and Customers using Spring Security.
- **Dynamic Membership Tiers:** Logic-based system for "Basic," "Premium," and "VIP" plans with automated start/end date calculations.
- **Equipment Management:** Real-time tracking of gym machinery status (Available, In-Use, Maintenance).
- **Session Booking System:** An interface for members to view available classes and book them instantly.
- **Live Audit Trail:** A persistent transaction history that logs every login, registration, and booking for security transparency.

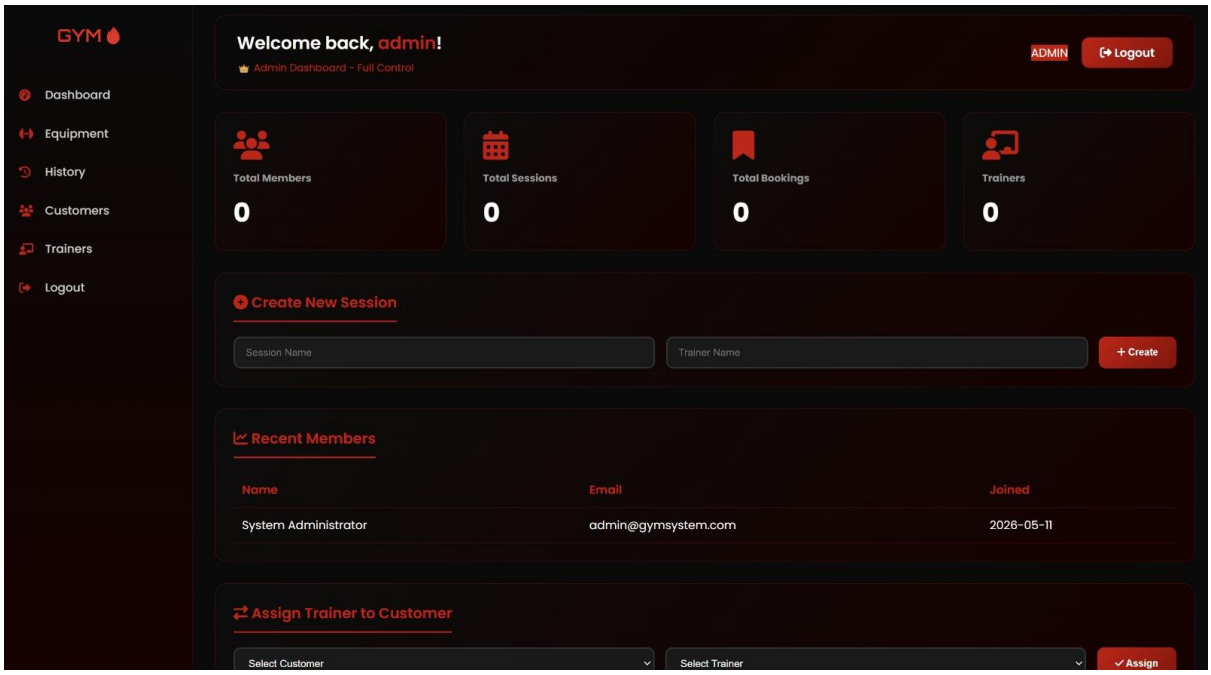
Figure 1: The secure login portal of GYMSystem.



3. User Roles & Permissions

- ***Administrator (ADMIN):*** *Full system access. Can view all registered customers, monitor every transaction, and manage the equipment database.*
- ***Professional Trainer (TRAINER):*** *Focuses on the "floor." Can manage session schedules, receive booking notifications, and update machine status.*
- ***Customer (USER):*** *Focuses on the "fitness journey." Can browse sessions, book trainers, and manage their personal membership status and history.*

Figure 2: The Admin Dashboard showing Total Members, Sessions, and Bookings.



4. Database Schema

User Table: Stores credentials, roles, and physical metrics (Weight/Height) or professional metrics (Specialization).

Membership Table: Tracks the type, price, and active/expired status of user plans.

Transaction Table: Records a timestamped log of every user action.

CREATE TABLE **users** (

id BIGINT PRIMARY KEY AUTO_INCREMENT,

full_name VARCHAR(255),

username VARCHAR(255),

email VARCHAR(255),

phone VARCHAR(255),

```
    password VARCHAR(255),
    role VARCHAR(50),         -- ROLE_ADMIN, ROLE_USER, ROLE_TRAINER
    user_type VARCHAR(50),    -- CUSTOMER, TRAINER
    -- Customer fields
    age INT,
    weight DOUBLE,
    height INT,
    fitness_goal VARCHAR(255),
    -- Trainer fields
    specialization VARCHAR(255),
    certification VARCHAR(255),
    experience_years INT,
    bio TEXT,
    joined_date VARCHAR(100)
);
CREATE TABLE booking (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(255),
    session VARCHAR(255),
    trainer VARCHAR(255)
);
CREATE TABLE session (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255),
```

```
    trainer VARCHAR(255)
);
CREATE TABLE machine (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255),
    type VARCHAR(255),            -- CARDIO, etc.
    status VARCHAR(255),          -- AVAILABLE, MAINTENANCE, IN_USE
    assigned_trainer VARCHAR(255)
);
CREATE TABLE membership (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(255),
    type VARCHAR(255),            -- BASIC, PREMIUM, VIP
    price DOUBLE,
    start_date DATETIME,
    end_date DATETIME,
    status VARCHAR(255)           -- ACTIVE, EXPIRED, CANCELLED
);
CREATE TABLE transaction_history (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(255),
    action VARCHAR(255),
    details TEXT,
    timestamp DATETIME
```

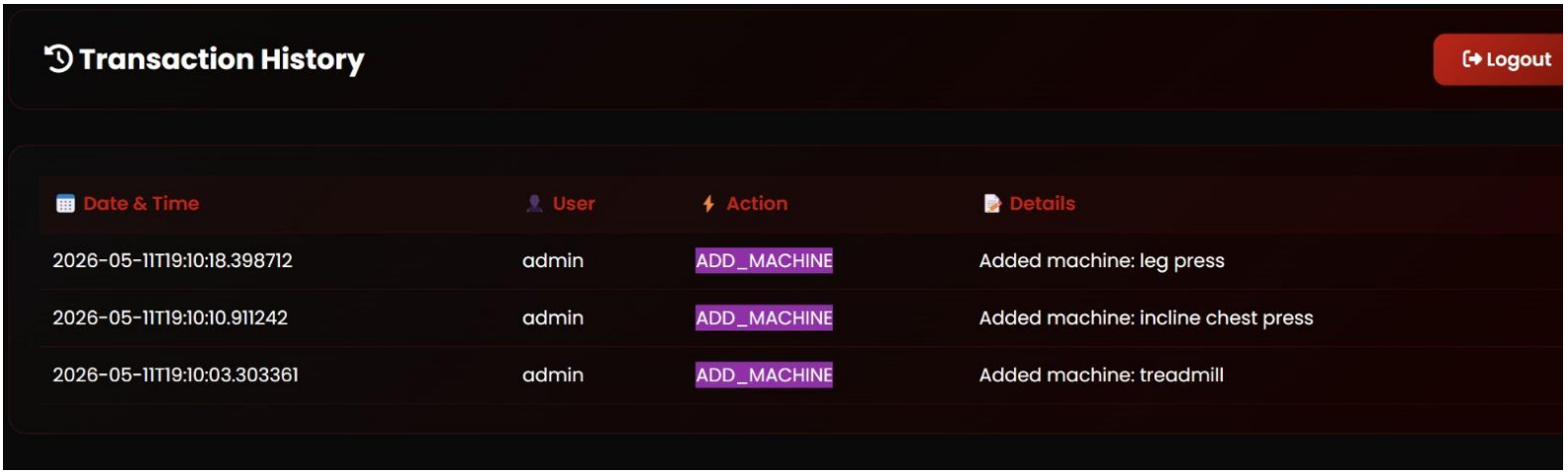
```
);

CREATE TABLE password_reset_token (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    token VARCHAR(255) UNIQUE,
    username VARCHAR(255),
    expiry_date DATETIME,
    used BOOLEAN
);
```

5. Process Flow (Sequence)

When a user interacts with the system (e.g., booking a session), the flow follows this sequence:

1. **UI:** User selects a session and clicks "Book."
2. **Controller:** The GymController receives the request and identifies the user via Principal.
3. **Service:** GymService validates if the user has an active membership.
4. **Repository:** If valid, the BookingRepo saves the record and TransactionHistoryRepo logs the event.
5. **UI:** The dashboard refreshes with a success message.

Figure 4: The system audit log showing recent transactions and user activity.



6. Technical Stack

- **Backend:** Java 17, Spring Boot 3.2.0.
- **Security:** Spring Security (BCrypt Password Encoding).
- **Frontend:** Thymeleaf, CSS3, FontAwesome.

UML DESIGN